

Distributed Representations of Words and Phrases and their Compositionality

📅 Date	May 19, 2026
☰ Multi-select	#NLP



arxiv.org

<https://arxiv.org/pdf/1310.4546>

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

2. Related Work 관련 연구

3. 제안 방법론

1. Skip-gram Model

1.1 계층적 소프트맥스(Hierarchical Softmax)

1.2 네거티브 샘플링(Negative Sampling, NEG)

1.3 Subsampling of Frequent Words

2. Learning Phrases 구문학습

4. 실험 및 결과

4.1 실험

Empirical Results - 3

Phrase Skip-Gram Results - 4.1

4.2 결과

Additive Compositionality - 5

Comparison to Published Word Representations -6

5. 결론 (배운점)

1. Introduction

논문이 다루는 분야

자연어 처리(NLP) 분야에서 단어와 구문(phrase)을 벡터 공간의 분산 표현(distributed representations)으로 학습시키는 방법

해당 task에서 기존 연구 한계점

- 기존의 신경망 아키텍처는 단어 벡터 학습 시 조밀한 행렬 곱셈을 포함하여 계산 비용이 매우 큼
- 기존의 단어 표현 방식은 단어의 순서를 무시하며, "Air Canada"와 같이 개별 단어의 의미 합으로 설명되지 않는 관용적 구문을 표현하지 못하는 한계
- 기본적인 Skip-gram 방식조차 어휘 사전의 크기가 커지면 소프트맥스(softmax) 계산량이 너무 많아져 비효율적임

논문의 contributions

- **Skip-gram 모델의 확장:**
학습 품질과 속도를 모두 향상시키는 여러 확장 기법을 제안
- **Negative Sampling:**
복잡한 계층적 소프트맥스를 대체할 수 있는 더 단순하고 효율적인 학습 알고리즘 제시
- **빈번한 단어의 Subsampling:** 매우 흔한 단어(예: "the", "in")를 학습에서 제외하여 속도를 높이고 희귀 단어의 학습 품질을 개선함
- **Phrase Learning:** 구문을 개별 토큰으로 취급하여 학습함으로써 "Boston Globe"와 같은 고유 명칭의 의미를 정확히 포착함
- **Additive Compositionality:** 벡터 간의 단순한 덧셈을 통해 의미 있는 복합 개념을 생성할 수 있음을 보여줌
 - 예를 들어, 벡터 계산 $\text{vec}(\text{"Madrid"}) - \text{vec}(\text{"Spain"}) + \text{vec}(\text{"France"})$ 의 결과는 다른 어떤 단어 벡터보다 $\text{vec}(\text{"Paris"})$ 에 더 가까움
 - $\text{vec}(\text{"Russia"}) + \text{vec}(\text{"river"})$ 는 $\text{vec}(\text{"Volga River"})$ 와 가깝고, $\text{vec}(\text{"Germany"}) + \text{vec}(\text{"capital"})$ 은 $\text{vec}(\text{"Berlin"})$ 과 가까움

기존보다 훨씬 빠른 방법으로 단어와 구문을 정확한 좌표로 옮김 → 컴퓨터가 단어 산수를 할 수 있을 정도로 똑똑해짐

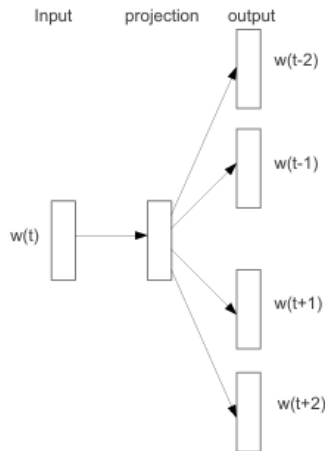


Figure 1: The Skip-gram model architecture. The training objective is to learn word vector representations that are good at predicting the nearby words.

2. Related Work 관련 연구

- 단어의 분산 표현은 1986년 Rumelhart 등에 의해 처음 제안되었으며, 이후 통계적 언어 모델링에 성공적으로 적용되어옴
- 최근 Mikolov 등(2013)은 대규모 비정형 데이터에서 고품질 벡터를 효율적으로 학습하는 Skip-gram 모델을 도입함
- 계층적 소프트맥스(Hierarchical Softmax)는 Morin과 Bengio에 의해 제안되어 계산 효율성을 높이는 데 사용됨
- Noise Contrastive Estimation(NCE)은 데이터와 노이즈를 구별하는 방식으로 모델을 학습시키는 기법으로 본 논문에서 제안한 네거티브 샘플링의 모태가 됨

3. 제안 방법론

1. Skip-gram Model

주어진 단어를 통해 주변의 문맥 단어들을 예측하도록 학습하는 모델

- 학습 목표는 문장이나 문서 내에서 주변 단어를 예측하는데 유용한 단어 표현 벡터를 찾는 것. $\Rightarrow$ 학습 단어의 시퀀스 $w_1, w_2, \dots, w_T$ 가 주어졌을 때, skip-gram 모델의 목적은 평균 로그 확률을 최대화하는 것
- 기본적 공식은 softmax 함수 사용 $\rightarrow$ 단어 예측 확률 정의
 - 어휘사전의 크기(W)가 커질경우, 계산비용이 커서 실용적X

1.1 계층적 소프트맥스(Hierarchical Softmax)

- 어휘 사전을 binary Huffman tree로 구성하여, W 개의 단어를 계산하는 대신 $\log_2(W)$ 개의 노드만 평가하도록 하여 속도를 개선함
- 계산 효율성을 높이기 위한 근사 방식
 - 어휘사전의 단어들을 앞으로 하는 이진 트리 구조, 루트에서 해당 단어까지 이르는 경로로 확률을 계산. 빈번한 단어에 짧은 코드를 할당해 학습 속도를 높이는 binary Huffman tree

$$p(w|w_I) = \prod_{j=1}^{L(w)-1} \sigma \left(\mathbb{I}[n(w, j+1) = \text{ch}(n(w, j))] \cdot v'_{n(w, j)}{}^\top v_{w_I} \right)$$

1.2 네거티브 샘플링(Negative Sampling, NEG)

모든 단어에 대한 확률을 계산하는 대신, 정답 단어와 소수의 노이즈 단어들만 구별하도록 학습하는 방식. 데이터셋이 클 경우 2~5개의 네거티브 샘플만으로도 효과적임

- 계층적 소프트맥스의 대안
- 정답 단어와 노이즈 분포에서 뽑은 k 개 가짜 단어들을 로지스틱 회귀로 구별하도록 학습
 - 노이즈 분포로는 단어 빈도수의 $3/4$ 승 ($U(w)^{3/4}$) 사용 시 성능이 좋았음

$$\log \sigma(v'_{w_O}{}^\top v_{w_I}) + \sum_{i=1}^{\kappa} \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-v'_{w_i}{}^\top v_{w_I}) \right]$$

- 💡 전체 사전에서 정답을 고르는 대신, 진짜 정답 1개와 무작위로 뽑은 오답(가짜) 5개를 놓고, 무엇이 진짜인지 맞추게 하는 시험
⇒ 문제가 훨씬 단순해지기 때문에 학습 속도가 비약적으로 빨라짐

- 국가(Country)와 해당 국가의 수도(Capital City) 사이의 관계
- 학습 과정에서 '수도'가 무엇인지에 대한 별도의 지도 학습을 제공하지 않았음에도, 모델이 스스로 개념을 조직화하고 단어 간의 관계를 암시적으로 학습할 수 있음
- 그래프를 보면 국가 간의 위치 관계와 수도 간의 위치 관계가 일정한 선형 구조를 보이며 배치됨

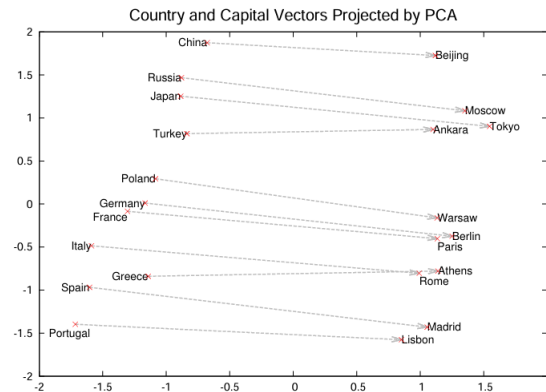


Figure 2: Two-dimensional PCA projection of the 1000-dimensional Skip-gram vectors of countries and their capital cities. The figure illustrates ability of the model to automatically organize concepts and learn implicitly the relationships between them, as during the training we did not provide any supervised information about what a capital city means.

1.3 Subsampling of Frequent Words

"the", "a"와 같이 정보 가치가 낮은 빈번한 단어들을 일정 확률로 학습에서 제외하여 학습 속도를 2~10배 높이고 희귀 단어의 정확도를 향상시킴

- 각 단어(w_i)의 특정 확률 $P(w_i)$ 에 따라 학습에서 제외하는 서브샘플링 기법 사용 → 빈도가 높은 단어는 과감히 제거하면서 빈도 순위는 유지
 - 학습 속도 높이고 드문 단어의 벡터 정확도도 향상시킴

2. Learning Phrases 구문학습

단순한 데이터 기반 접근법을 사용하여 자주 같이 등장하지만 다른 문맥에서는 드물게 나타나는 단어 쌍(예: "New York Times")을 하나의 토큰으로 결합

- 단어들의 단순 합으로 의미 파악할 수 없는 관용적 구문 처리
- 구문의 정의와 필요성: 개별 단어의 의미를 합친것과 완전히 다른 독자적 의미를 가지는 구문 → 개별적 하나의 토큰으로 취급
- 구문을 찾기 위해 unigram과 bigram의 빈도를 이용한 통계적 접근법 사용
- 구문 점수(score) 공식: 두 단어 w_i 와 w_j 가 하나의 구문을 이룰 가능성을 아래 공식으로 계산
 - δ (델타)는 discounting coefficient로, 아주 드물게 등장하는 단어들이 우연히 같이 나타나서 구문으로 오인되는 것을 방지

- 설정된 임계값(threshold) 이상의 점수를 가진 바이그램만 구문으로 병합

$$\text{score}(w_i, w_j) = \frac{\text{count}(w_i w_j) - \delta}{\text{count}(w_i) \times \text{count}(w_j)}.$$

- 데이터셋에 대해 임계값을 낮추어 가며 2~4회 반복(pass)해서 훑습니다. 이를 통해 두 단어로 된 구문뿐 아니라 여러 단어가 결합된 더 긴 구문도 형성할 수 있음

4. 실험 및 결과

4.1 실험

Empirical Results - 3

계층적 소프트맥스(HS), 노이즈 대비 추정(NCE), 네거티브 샘플링(NEG), 그리고 단어 서브샘플링의 성능을 평가

- **평가 방법:** "독일 : 베를린 :: 프랑스 : ?"와 같은 단어 유추 작업(Analogical Reasoning Task)을 사용. 이 문제는 `vec("베를린") - vec("독일") + vec("프랑스")`를 계산했을 때 `vec("파리")`가 나오면 정답으로 인정
- **데이터:** 10억 개에서 최대 300억개의 단어로 구성된 Google 뉴스 데이터셋을 사용했으며, 어휘 사전은 약 69만 개
- **실험 결과:**
 - 네거티브 샘플링(NEG)이 계층적 소프트맥스(HS)보다 성능이 뛰어났으며, NCE보다도 약간 더 나은 결과
 - **서브샘플링**을 적용했을 때 학습 속도가 수 배 빨라졌을 뿐만 아니라, 단어 표현의 정확도도 훨씬 높아짐
 - Skip-gram 모델의 선형성(Linearity) 덕분에 이러한 선형적인 유추 문제가 아주 효과적으로 해결됨

💡 진짜와 가짜를 구별하는 공부법(네거티브 샘플링)이 스무고개 방식(계층적 소프트맥스)보다 더 똑똑한 결과를 냄

Method	Time [min]	Syntactic [%]	Semantic [%]	Total accuracy [%]
NEG-5	38	63	54	59
NEG-15	97	63	58	61
HS-Huffman	41	53	40	47
NCE-5	38	60	45	53
The following results use 10^{-5} subsampling				
NEG-5	14	61	58	60
NEG-15	36	61	61	61
HS-Huffman	21	52	59	55

Table 1: Accuracy of various Skip-gram 300-dimensional models on the analogical reasoning task as defined in [8]. NEG- k stands for Negative Sampling with k negative samples for each positive sample; NCE stands for Noise Contrastive Estimation and HS-Huffman stands for the Hierarchical Softmax with the frequency-based Huffman codes.

Phrase Skip-Gram Results - 4.1

구문 학습의 효과를 검증하기 위해 뉴스 데이터를 활용해 다양한 하이퍼파라미터로 실험

Newspapers			
New York San Jose	New York Times San Jose Mercury News	Baltimore Cincinnati	Baltimore Sun Cincinnati Enquirer
NHL Teams			
Boston Phoenix	Boston Bruins Phoenix Coyotes	Montreal Nashville	Montreal Canadiens Nashville Predators
NBA Teams			
Detroit Oakland	Detroit Pistons Golden State Warriors	Toronto Memphis	Toronto Raptors Memphis Grizzlies
Airlines			
Austria Belgium	Austrian Airlines Brussels Airlines	Spain Greece	Spainair Aegean Airlines
Company executives			
Steve Ballmer Samuel J. Palmisano	Microsoft IBM	Larry Page Werner Vogels	Google Amazon

Table 2: Examples of the analogical reasoning task for phrases (the full test set has 3218 examples). The goal is to compute the fourth phrase using the first three. Our best model achieved an accuracy of 72% on this dataset.

- **실험 설정:** 300차원의 벡터, 문맥 크기(window size) 5를 기본으로 설정하여 네거티브 샘플링(NEG)과 계층적 소프트맥스(HS)를 비교
- **주요 실험 결과 (Table 3 기준):**

Method	Dimensionality	No subsampling [%]	10^{-5} subsampling [%]
NEG-5	300	24	27
NEG-15	300	27	42
HS-Huffman	300	19	47

Table 3: Accuracies of the Skip-gram models on the phrase analogy dataset. The models were trained on approximately one billion words from the news dataset.

- **네거티브 샘플링(NEG):** 오답 샘플 수 $k=5$ 일 때보다 $k=15$ 일 때 성능이 훨씬 좋았음 (서브샘플링 적용 시 27% → 42%)
- **서브샘플링 효과:** 서브샘플링이 없을 때는 계층적 소프트맥스(19%)가 네거티브 샘플링(24~27%)보다 뒤처졌으나, 서브샘플링을 적용하자 계층적 소프트맥스(47%)가 모든 방식 중 가장 높은 성능을 보임
- **데이터 규모의 위력:**
 - 학습 데이터를 330억 개의 단어로 늘리고, 1000차원 벡터와 전체 문장을 문맥으로 사용했을 때 72%라는 최고 정확도를 달성함
 - 데이터를 60억 개로 줄이면 정확도가 66%로 떨어짐 → 구문 학습에는 방대한 양의 데이터가 필수적임을 확인
- **질적 평가 (Table 4):** 희귀한 구문의 '가장 가까운 이웃' 단어들을 분석했을 때, 수치적 결과와 마찬가지로 계층적 소프트맥스에 서브샘플링을 결합한 모델이 가장 의미론적으로 정확한 구문 벡터를 생성하는 것으로 나타남

	NEG-15 with 10^{-5} subsampling	HS with 10^{-5} subsampling
Vasco de Gama	Lingsugur	Italian explorer
Lake Baikal	Great Rift Valley	Aral Sea
Alan Bean	Rebecca Naomi	moonwalker
Ionian Sea	Ruegen	Ionian Islands
chess master	chess grandmaster	Garry Kasparov

Table 4: Examples of the closest entities to the given short phrases, using two different models.

단순한 통계 공식을 반복 적용하여 구문을 찾아내는 효율적인 방법을 제안 → 실험을 통해 대규모 데이터와 적절한 기법(HS + 서브샘플링)이 결합될 때 구문의 의미를 가장 완벽하게 포착할 수 있음을 입증

4.2 결과

Additive Compositionality - 5

Skip-gram 모델로 학습된 벡터들이 단순한 요소별 덧셈(element-wise addition)을 통해 의미 있는 결합을 이룰 수 있다는 독특한 특성

- 단어와 구문 벡터들이 선형 구조를 가지고 있어, 벡터 산술 연산을 통해 정교한 유추가 가능할 뿐만 아니라 단어들을 의미 있게 결합할 수도 있음
- **주요 예시 (Table 5 참조):**

Czech + currency	Vietnam + capital	German + airlines	Russian + river	French + actress
koruna	Hanoi	airline Lufthansa	Moscow	Juliette Binoche
Check crown	Ho Chi Minh City	carrier Lufthansa	Volga River	Vanessa Paradis
Polish zolty	Viet Nam	flag carrier Lufthansa	upriver	Charlotte Gainsbourg
CTK	Vietnamese	Lufthansa	Russia	Cecile De

Table 5: Vector compositionality using element-wise addition. Four closest tokens to the sum of two vectors are shown, using the best Skip-gram model.

- $\text{vec}(\text{"Russia"}) + \text{vec}(\text{"river"}) \approx \text{vec}(\text{"Volga River"})$
- $\text{vec}(\text{"Germany"}) + \text{vec}(\text{"capital"}) \approx \text{vec}(\text{"Berlin"})$
- $\text{vec}(\text{"Czech"}) + \text{vec}(\text{"currency"}) \approx \text{vec}(\text{"koruna"})$
- **원리:** 단어 벡터는 소프트맥스 비선형성 입력과 선형 관계에 있음. 벡터들은 단어가 나타나는 문맥의 분포를 나타내는데, 두 벡터를 더하는 것은 두 문맥 분포의 곱(product)과 관련 있음. 이 곱은 'AND' 함수처럼 작용하여, 두 단어 벡터 모두에서 높은 확률을 부여받은 단어(예: "Volga River"는 "Russia"와 "river" 문맥 모두에서 자주 등장)가 결과 벡터에서도 높은 확률을 갖게 됨

Comparison to Published Word Representations -6

Collobert와 Weston, Turian 등 기존의 유명한 연구자들이 공개한 단어 벡터들과 성능을 비교

- 이전 연구들(Mikolov et al., 2013)의 평가에서도 이미 Skip-gram 모델이 단어 유추 작업에서 다른 모델들을 큰 차이로 앞선 바 있다

- **희귀 단어 표현력** (Table 6 참조): "Redmond"나 "ninjutsu"와 같이 자주 나오지 않는 단어들의 '가장 가까운 이웃'을 찾았을 때, 기존 모델들은 관련성이 낮은 단어들을 제시하는 반면, 본 논문의 Skip-phrase 모델은 "Redmond Wash.", "martial arts" 등 매우 정확한 연관 단어를 찾아냄
- **성공 요인:**
 - **데이터의 양:** 기존 연구보다 2~3개 차수(orders of magnitude) 더 많은 **330억 개의 단어**를 학습
 - **효율성:** 학습 데이터는 훨씬 방대해졌음에도 불구하고, 모델 아키텍처가 워낙 효율적이어서 학습 시간은 오히려 기존 모델들의 극히 일부분밖에 걸리지 않음

Model (training time)	Redmond	Havel	ninjutsu	graffiti	capitulate
Collobert (50d) (2 months)	conyers lubbock keene	plauen dzerzhinsky osterreich	reiki kohona karate	cheesecake gossip dioramas	abdicate accede rearm
Turian (200d) (few weeks)	McCarthy Alston Cousins	Jewell Arzu Ovitz	- - -	gunfire emotion impunity	- - -
Mnih (100d) (7 days)	Podhurst Harlang Agarwal	Pontiff Pinochet Rodionov	- - -	anaesthetics monkeys Jews	Mavericks planning hesitated
Skip-Phrase (1000d, 1 day)	Redmond Wash. Redmond Washington Microsoft	Vaclav Havel president Vaclav Havel Velvet Revolution	ninja martial arts swordsmanship	spray paint graffiti taggers	capitulation capitulated capitulating

Table 6: Examples of the closest tokens given various well known models and the Skip-gram model trained on phrases using over 30 billion training words. An empty cell means that the word was not in the vocabulary.

5. 결론 (배운점)

1. Skip-gram 모델을 사용하여 단어와 구문의 분산 표현을 학습하는 방법을 제시하고, 이들이 선형 구조를 가짐을 입증함
2. 계산 효율성 덕분에 기존보다 수백 배 많은 데이터를 학습시켜 특히 rare entities에 대한 벡터 품질을 획기적으로 높임
 - a. 계산 효율적인 아키텍처를 통해 기존보다 수백 배 많은 데이터를 학습시키는 것이 벡터의 품질, 특히 희귀한 개체에 대한 표현력을 높이는 핵심
3. Negative Sampling이라는 매우 단순하면서도 정확한 학습 알고리즘을 제안
4. Subsampling을 통해 학습 속도 향상 + 드문 단어의 정확도 개선

- 성능에 가장 큰 영향을 미치는 결정 요소(하이퍼파라미터)는 모델 아키텍처, 벡터 크기, 서브샘플링 비율, 그리고 윈도우 크기임을 확인
- 복잡한 행렬 연산 없이 구문을 단일 토큰으로 처리하거나 단순 덧셈을 사용하는 것만으로도 긴 텍스트 표현



더 단순한 수학과 더 방대한 데이터가 복잡한 인공지능 모델보다 언어를 더 잘 이해할 수 있음을 보여주었으며, 현대 자연어 처리 기술의 기초가 된 `word2vec`의 토대를 마련함

네거티브 샘플링과 같은 단순한 알고리즘과 벡터 가산(addition)과 같은 기본적인 수학 연산만으로도 복잡한 언어적 구조와 고차원적인 의미 이해가 가능하다는 점이 놀라움